

RESEARCH

Leanstral 1.5: Proof Abundance for All

July 2, 2026

By Leanstral Team at Mistral AI

 [Back to Blog](#)

6 MIN READ

Share this post 



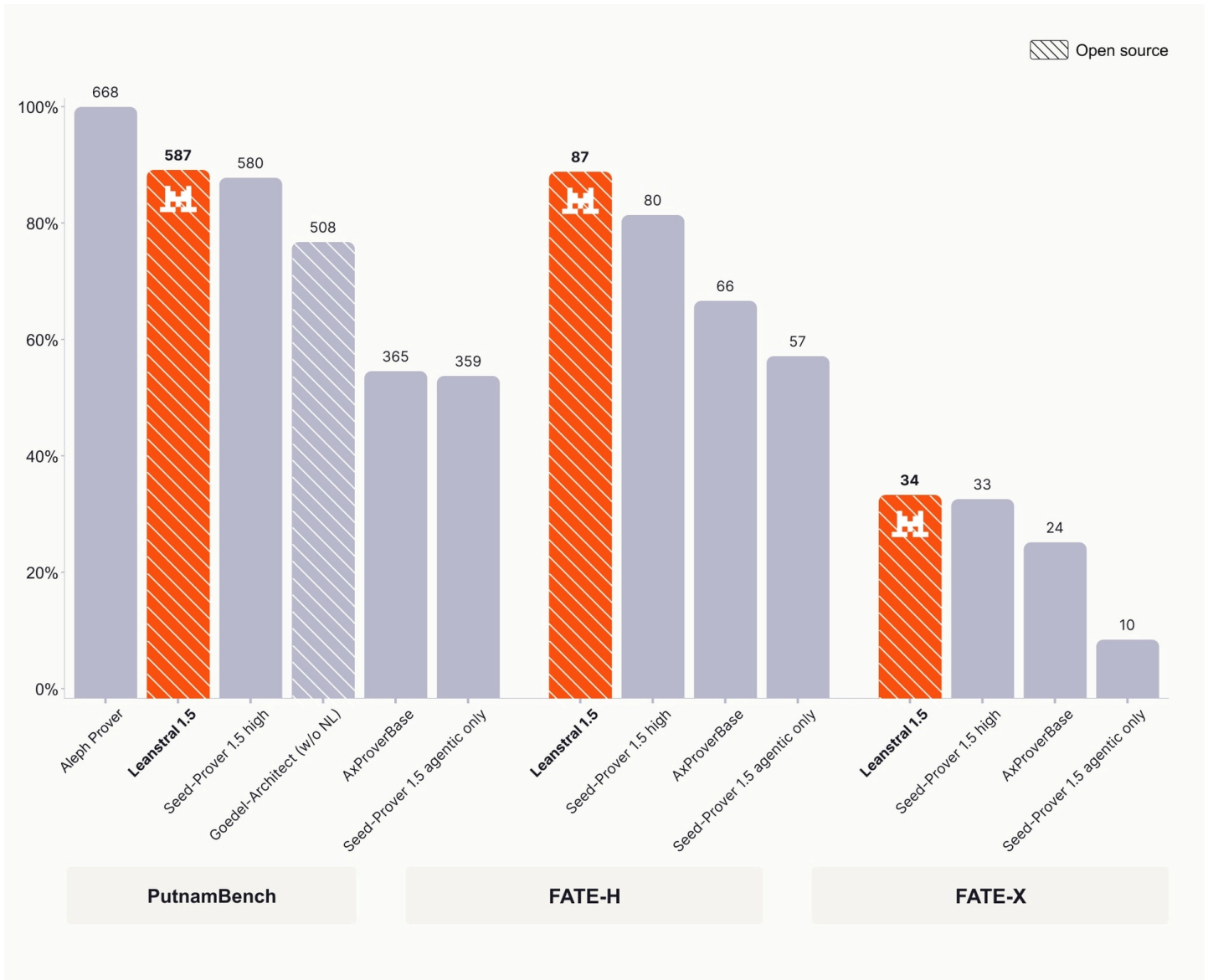
Summary

Leanstral 1.5, a free Apache-2.0 licensed model with 6B active parameters, delivers a major performance upgrade in formal verification, saturating miniF2F, solving 587/672 PutnamBench problems, and achieving state-of-the-art results on FATE-H (87%) and FATE-X (34%). Trained through mid-training, supervised fine-tuning, and reinforcement learning with CISPO, it excels in agentic proof engineering and real-world code verification, uncovering 5 previously unknown bugs across 57 repositories tested. Fully open-sourced and available via Hugging Face and a free API, Leanstral 1.5 is now accessible for practical proof engineering in Lean 4.

Since its launch, Leanstral has offered an open, practical approach to proof engineering in [Lean 4](#). Today, we are releasing **Leanstral 1.5**, a free Apache-2.0 licensed model with 119B total and only 6B active parameters, delivering a performance upgrade that makes formal verification more powerful and accessible than ever.

Leanstral 1.5 **saturates miniF2F**, solves **587/672 PutnamBench problems**, and achieves a new state-of-the-art of **%87 on FATE-H** and **34% on FATE-X**. Beyond benchmarks, it verifies complex code properties and **uncovers previously unknown bugs in open-source repositories**—proving that rigorous formal methods can be both effective and practical for real-world use.

	Products	Solutions	Research	Developers	Blog	Customers	Company
--	----------	-----------	----------	------------	------	-----------	---------

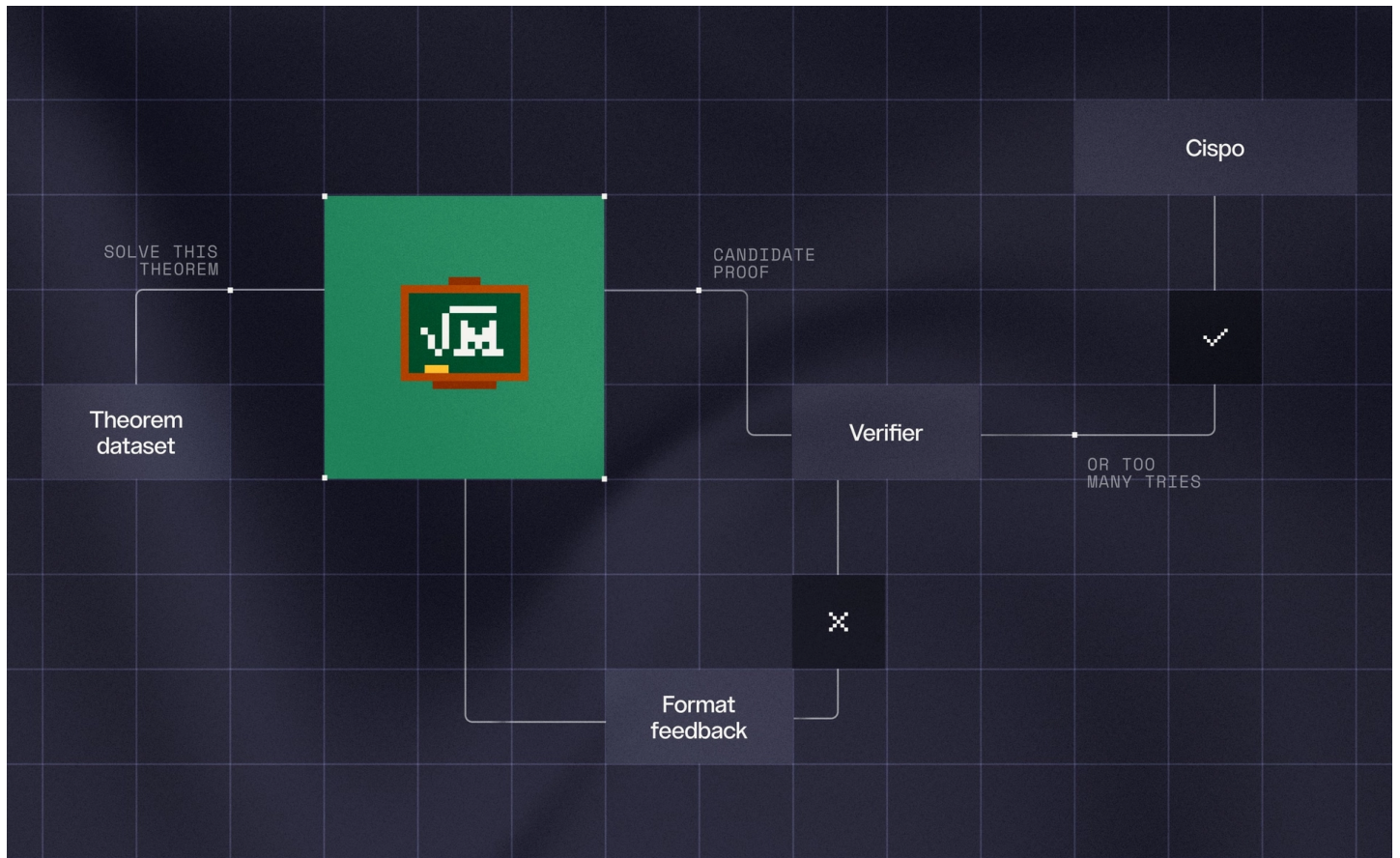


Training Leanstral

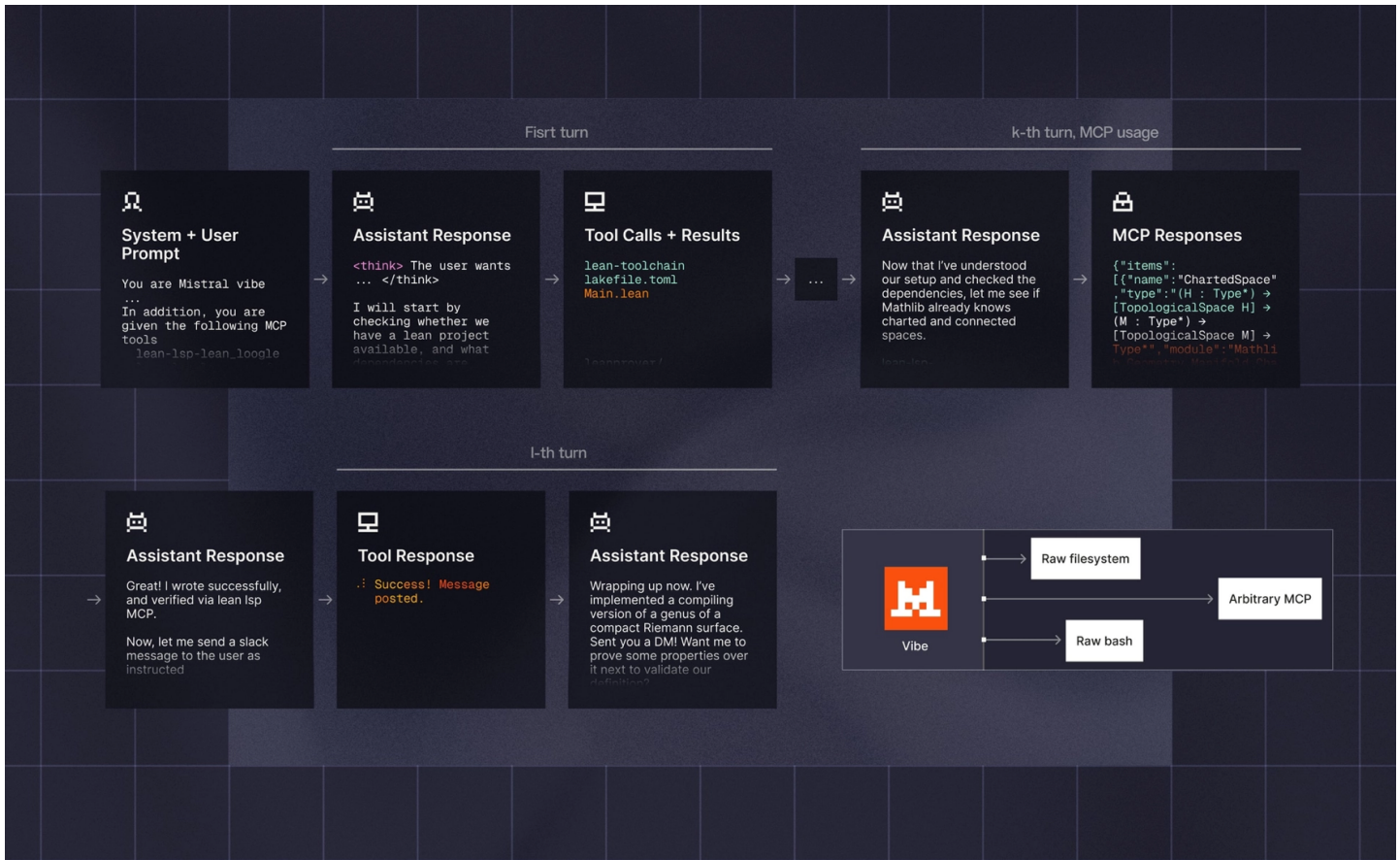
Leanstral 1.5 goes through a three-stage process: mid-training, supervised fine-tuning, and reinforcement learning with CISPO. Leanstral 1.5 leverages extensive training on two RL environments:

In the **multiturn environment**, the model is given a theorem statement and

must either prove or disprove it. The model submits a proof, receives Lean compiler feedback, and refines its approach with each attempt. If the proof compiles it succeeds; otherwise the loop continues until the model either solves the problem or exhausts its budget.



In the **code agent environment**, Leanstral operates like a developer in a raw filesystem: it edits files, runs bash commands, and uses the Lean language server to inspect goals, errors, and type information in real time. This allows it to tackle long-horizon tasks like completing partial proofs in a repository, building auxiliary lemmas, and persisting through multiple rounds of context compaction. The model learns to navigate the full proof-engineering workflow and is finally verified by our fork of SafeVerify for correctness given a list of target theorems.



Evaluation

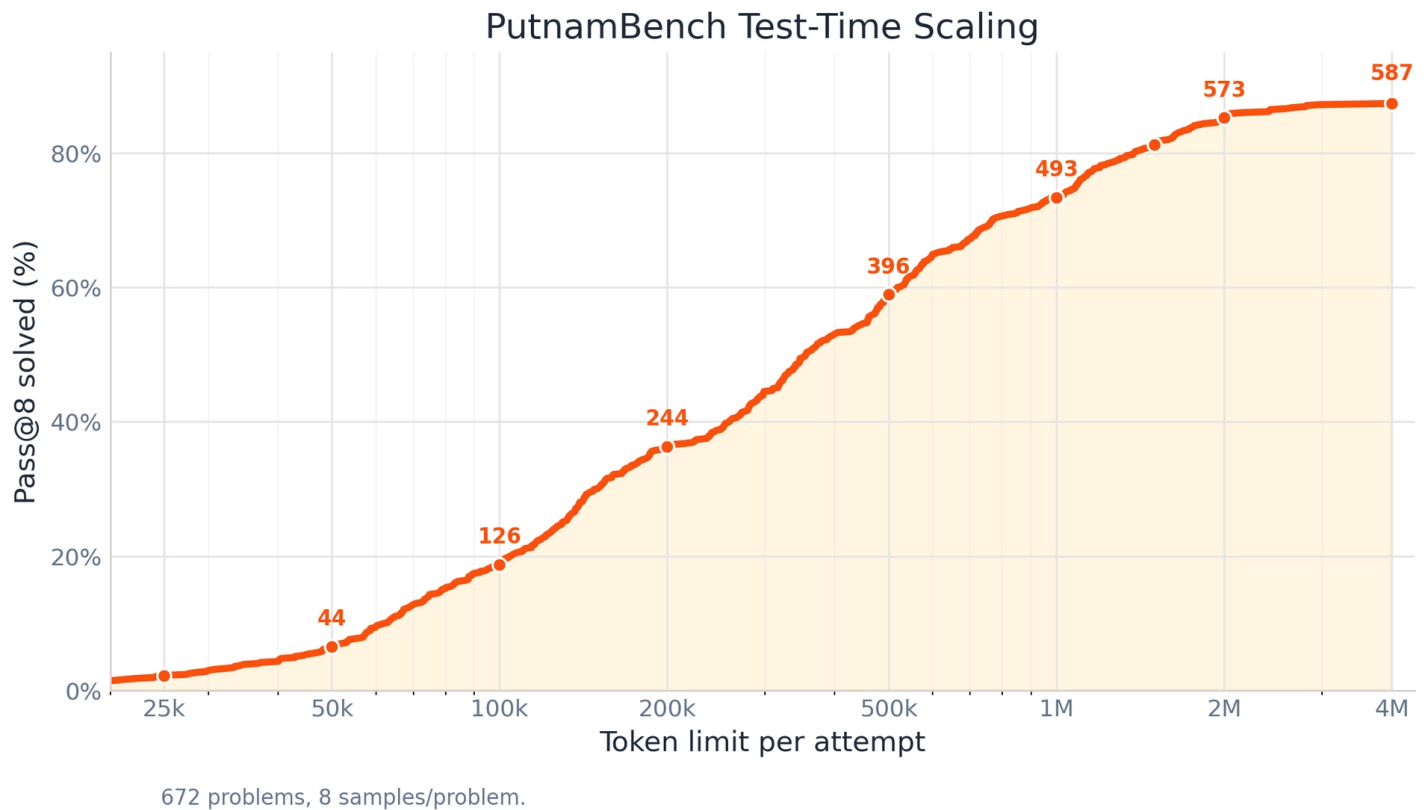
We evaluate Leanstral on the following benchmarks:

- **miniF2F** is a cross-system benchmark for formal mathematics, ranging from elementary problems to IMO-level challenges, testing diverse proof abilities across algebra, combinatorics, and number theory.
- **PutnamBench** consists of 672 problems from the Putnam Mathematical Competition, requiring deep reasoning and long proof chains to solve challenging mathematical problems.
- **FATE-H and FATE-X** are abstract algebra benchmarks for graduate and PhD-level problems, respectively, testing advanced reasoning in areas like group theory, ring theory, and module theory.
- **FLTEval** is based on real pull requests from the Fermat's Last Theorem repository, testing practical proof engineering with real-world complexity.

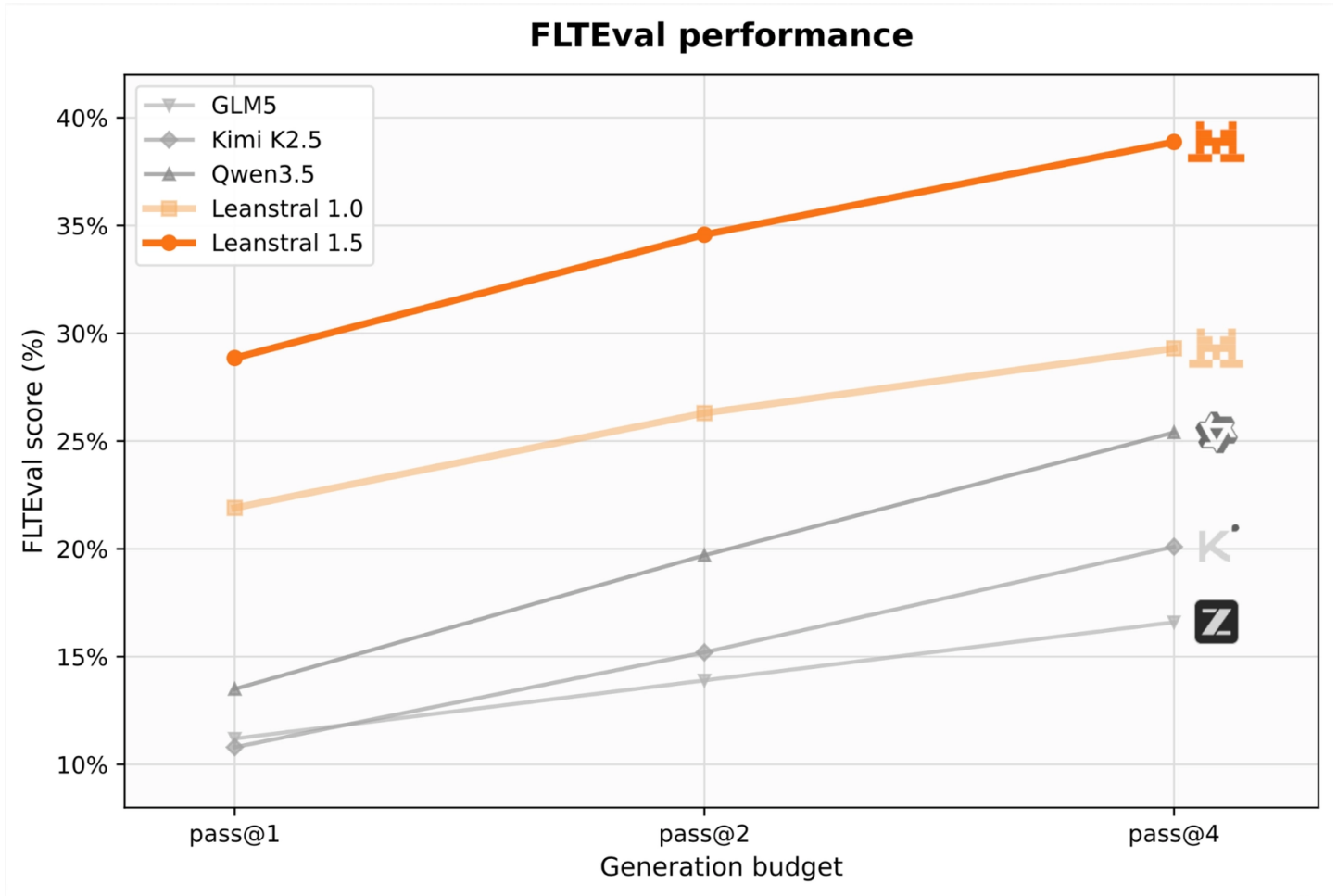
We saturate miniF2F completely, reaching 100% on both the validation and test sets. On PutnamBench and FATE-H/X, we compare Leanstral 1.5 against Goedel-Architect without natural-language guidance, Seed-Prover 1.5 at its high setting, and AxProverBase. Leanstral reaches a new state-of-the-art on FATE-H/X, solving 87 and 34 problems respectively. On PutnamBench, it edges out Seed-Prover 1.5 high by 7 problems at far lower cost: about \$4 per problem, against an estimated \$300 or more for Seed-Prover, whose high setting runs with a budget of 10 H20-days per problem. The only provers ranked higher operate under different conditions—some receive natural-language proof guidance, others cost far more to run, like Aleph Prover at \$54–68 per problem.

Leanstral 1.5 shows the strongest test-time scaling we have seen from a formal-reasoning model. The figure below tracks Pass@8 on PutnamBench as we raise the token budget per attempt from 25k to 4M: performance

climbs smoothly and monotonically the whole way, from 44 problems solved at 50k to 244 at 200k, 493 at 1M, and 587 at 4M. Rather than giving up when a proof runs long, Leanstral keeps reasoning, editing files, and revising across millions of tokens, turning that budget directly into solved problems—the same behavior behind the AVL-tree proof below, which ran for over 2.7 million tokens across 22 compactions.



With this release, we also fully open source **FLTEval**. Leanstral 1.5 lifts pass@1 on the benchmark from 21.9 to 28.9 and pass@8 from 31.9 to 43.2, surpassing Opus 4.6's 39.6 at one-seventh the cost. It also widens its lead over open-source models 3–10× larger, as shown in the figure below.



Code Verification Case Studies

While being primarily trained for mathematics, Leanstral 1.5 exhibits strong abilities in code verification. We present 2 critical case studies to demonstrate its impact.

AVL Trees: Proving Time Complexity

AVL trees are self-balancing binary search trees that maintain $O(\log n)$ height through rebalancing during insertions and deletions. Leanstral 1.5 proved these time complexity guarantees for a real implementation—a task

that required structural induction to mirror the tree's recursive structure, careful handling of monadic time tracking, and exhaustive case analysis for rebalancing paths. Over 2.7 million tokens and 22 compactions, Leanstral systematically unfolded each layer of the TimeM monad, exposing the underlying computations despite their interleaving with control flow. It established an almost tight bound of 48 steps per height unit plus a constant for insertion, then connected height to tree size via a logarithmic relationship, delivering complete, verified proofs that insertion and deletion are indeed $O(\log n)$.

Bug Discovery: Finding Hidden Flaws

To test Leanstral's bug-catching abilities, we built an automated pipeline: Aeneas translates Rust code to Lean, while Leanstral infers the user intent and generates correctness properties from the code. Leanstral then attempts to prove each property in four attempts. If they all fail, it tries to prove the negation instead, also with four attempts. Across 57 tested repositories, this process flagged 47 violated properties, with 11 pointing to genuine bugs—5 of them previously unreported on GitHub.

One such bug was in the sign function for zigzag decoding of the **datrs/varinteger** library. On input `Std.U64.MAX`, the expression `(value + 1)` overflowed, causing crashes in debug mode and silent corruption in release mode—an edge case that testing and fuzzing would typically miss. Leanstral's pipeline caught it automatically, demonstrating that formal verification can already be applied to real-world codebases and find bugs that some traditional methods overlook.

Get Started

Leanstral 1.5 has a **Apache-2.0** license. The weights can be found [on Huggingface](#), while also being available now [as a free API endpoint](#) as `leanstral-1-5`. We recommend using it in Mistral Vibe. To begin your journey, grab an API Key, and:

1. Set up Mistral Vibe

```
uv tool install mistral-vibe
uv tool update mistral-vibe
vibe --setup
```

2. Install Leanstral 1.5

```
/leanstall
exit
```

3. Launch the agent

```
vibe --agent lean
```

4. Install Lean LSP MCP (Optional)

It is highly recommended to install **Lean LSP MCP** by adding the following to your `~/.vibe/config.toml`

```
[[mcp_servers]]
name = "lean-lsp"
transport = "stdio"
command = "uvx"
args = ["lean-lsp-mcp"]
tool_timeout_sec = 600
```

If there are no existing MCP servers, you may have to remove `mcp_servers = []`.

5. Start proving

Ask Leanstral to tackle a theorem, debug a proof, or contribute to a repository. It's that simple.

19%

Products	Solutions	Why Mistral	Legal
Vibe	Delivery	About us	Terms of Service
Vibe Code	methodology	Careers	Privacy Policy
Studio	Model	Partners	Privacy choices
Forge		Our customers	Data processing

Forge Compute Pricing	customization Coding Document intelligence Speech Mistral for finance Mistral for public institutions Mistral for manufacturing	Our customers Our models Brand	Data processing agreement Legal notice
---	---	--	---



Get Mistral Vibe



